

Today we deal with False Data Injection Attack (FDIA), which means some measurements are altered by false data. The reference scenario is given by the grid, from which we take measurements, which we call z (the vector of measurements).

These measurements go into the control center. The values first undergo a check, given by Bad Data Detector. After data is cleaned, it goes through a computer algorithm performing State Estimation. We call x the state of the grid. It's difficult to measure all x variables, so what we measure are the z variables that depend on x . As for the State Estimation, the goal is to find an estimate of the state, $\hat{x}$. After knowing the state, we can compute some control actions. We'll have many controllers. We then implement all these controls taken from controllers in the grid.

This is a feedback control loop.

The attacker will inject false data into the measurement. We call the vector a (as for attack), and what arrives in the control center $z_a = z + a$.

The goal of the attacker is to understand which sensors to target, and what values to inject. So the goal is twofold, given by:

1. Impact, altering the state estimation.

In nominal conditions, the state estimate $\hat{x}$ (cyber state), if everything goes, is practically close to the real state, so $\hat{x} \approx x$.

Under attack, the attacker wants it to be different from the real state, so $\hat{x}_a \not\approx x$, implying that decisions are wrong, and controllers using $\hat{x}_a$ will take wrong decisions on the grid. This will bring some damage. Typically, FDIA on state estimation is just the first step of a more complex attack, as the final goal is not just to alter the state estimation, but to alter it to produce a specific effect on the grid.

2. Remain undetected by BDD (Bad Data Detector).

The model of the grid will tell us what is the dependence between the state variables and the measurements. We have that z is a function of x , $z = h(x)$. Step two will be to study how state estimation works, and step 3 studying BDD. Finally, we'll see how to design an attack.

Imagine we have a grid, we are the operator and want to control this grid. The grid is made of several objects. The black line is the bus, while the circle with

something funny inside is the generator. Arrows are consumption. Nodes are geographical location of ?. The arrow typically tells the direction of power. Red boxes are switches, used to connect and disconnect the line. The topology of the grid is not fixed, and changed over the day. It could be changed for maintenance, or to optimize the grid.

The power flow equations describe how power flow from a bus i to bus j . What are important variables? One is P_{ij} , which is the power going from i to j . We also have the voltage amplitude and voltage phase at the bus, respectively V_j and Θ_j .

The equations are called line powerflow equations. They're not linear, as they depend on the cosine. $\Phi_{ij} = \Theta_i - \Theta_j$.

$$\begin{aligned} P_{ij} &= V_i^2 g_{ij} - V_i V_j \dots \\ Q_{ij} &= -V_i^2 \dots \end{aligned}$$

The rest of values are parameters. Every line has corresponding g, b parameters, depending on length, etc.

The state, typically, is given by voltage magnitudes and voltage angles. By knowing these two in busses, we can compute any power flows in the network. We need to ensure these powers stay between safety limits.

So, state x is given by voltage magnitude and angle (phase, shift) on all the busses. Why it's called state? By knowing them, we can compute all the power flows in the network, checking if they are in safety limits.

When we studied dynamical system, we defined what is a state. Here it's different, as we do not have differential equations. This is a static system.

It's difficult to directly measure voltage at every bus, so, instead, we collect power measurements, and from these measurements we estimate voltage.

We define z as P_{ij} and Q_{ij} roughly, telling us how the power is related with voltage. So, $z = h(x)$, and h is given by the power flow equations. The function h is non linear, and this problem could indeed be a nonlinear state estimation problem, but we focus instead on linear state estimation. Why? In many applications it makes sense to consider a linearized system. The idea is that, in transmission grids for example, voltage is more or less constant, typically at 1 p.u.. Values are typically normalized, and when we say 1 p.u. we mean that the voltage is at the reference.

So, we can consider all V_j at 1 p.u.. Also, we can ignore g , as it's very close to zero, and we have that Θ_{ij} is very small, meaning that $\sin(\Theta) = \Theta$ (can be

approximated to).

For a good approximation, we have that $P_{ij} = b_{ij}\Theta_{ij}$, with b_{ij} being a scalar, and this is equal to $= b_{ij}\Theta_i - b_{ij}\Theta_j$. In this model, P is a linear function of Θ . So, Θ is the state, and P are measurements.

Let's do a simple example.

Let's say we have three busses. To the first, a generator is connected, to the second nothing, and to the third we have a consumer. We define the generated power P_G , and P_L the power of the load. We have a connection from bus 1 to 2, and from 2 to 3.

Our state vector is thus given by:

$$x = \begin{pmatrix} \Theta_1 \\ \Theta_2 \\ \Theta_3 \end{pmatrix}$$

Depending on where we place sensors, we have a particular structure for the vector. Let's assume we have a sensor on the cable from 1 to 2, measuring P_{12} . We have then another meter in P_L , measuring $P_L = P_3$. Finally, let's say we also measure P_{23} .

In this example, $P_G = P_1 = P_{12}$. Similarly, $P_3 = P_L = P_{23}$.

Let's compute then z_1, z_2 and z_3 .

$$z = \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} P_{12} \\ P_{23} \\ P_{23} \end{pmatrix}$$

We have redundant meters in this case, in reality we would not have something like this.

We have, then

$$\begin{aligned} P_{12} &= b_{12}\Theta_1 - b_{12}\Theta_2 \\ P_{23} &= b_{23}\Theta_2 - b_{23}\Theta_3 \\ P_{23} &= b_{23}\Theta_2 - b_{23}\Theta_3 \end{aligned}$$

This is in scalar form, let's now put it in matrix form.

$$\begin{pmatrix} P_{12} \\ P_{23} \\ P_{23} \end{pmatrix} = z = \begin{pmatrix} b_{12} & -b_{12} & 0 \\ 0 & b_{23} & -b_{23} \\ 0 & b_{23} & -b_{23} \end{pmatrix} \begin{pmatrix} \Theta_1 \\ \Theta_2 \\ \Theta_3 \end{pmatrix} = Hx$$

H depends on:

- Parameters of the lines (b_{ij} coefficients);
- Where we place the sensors;
- Topology of the grid.

Let's once again consider the topology described above. By changing the topology, the H matrix changes as well. For example, what happens if we have another line from bus 1 to bus 3?

So, $z = Hx$ is the mathematical model linking measurements to state variables.

What happens when we take measurements from the grid?

We have the grid, in a given state x we do not know. We cannot measure voltage, but we get the Power, depending on voltage. We know that $z = Hx$, which is an approximation, but let's assume for now this is the exact model. We send these

measurements to the control center. Imagine $x = \begin{pmatrix} 0.1 \\ 0.05 \\ 0 \end{pmatrix}$. If we go inside the

control center and read measurements, which values do we expect to see? We expect to see, ideally:

$$z = \begin{pmatrix} H_1 x \\ H_2 x \\ H_3 x \end{pmatrix}$$

However, we will not see exactly this, as the measurement equation is given by:

$$z = Hx + e \quad e \text{ is the measurement noise}$$

Every sensor has an error, known as e .

We then have:

$$z = \begin{pmatrix} z_1 \\ \vdots \\ z_m \end{pmatrix} \in \mathbb{R}^m \quad x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n \quad H \in \mathbb{R}^{m \times n}$$

We have m sensors (power meters), and n are state variables, so a grid with n buses.

Now, when we measure things the sensor will not return the exact value, as it has a random value. Typically, the error is modelled as Gaussian curve. So, the expected value of the error is zero, so $E[e_i] = 0$. This is an unbiased sensor. How large is the error is given by the variance of the Gaussian error. The larger the variance, the larger is the curve. We want to work with sensors that most of the times give us small errors.

If we have a very narrow distribution, the probability of having a big error is small.

Every measurement is associated to a different error, and to know how good is the sensor we use $\delta_{e_i}^2$, which is known from the datasheet of sensors.

The measurement equations tells us how the measurements we have are linked to other variables we want to estimate. $z = Hx$ is the physical model of the system, with x being hidden variables we cannot directly measure, and when we measure we add the error, having $z = Hx + e$. The problem now is that we have z , and how do we find a good estimate of $\hat{x}$?

We consider that we have poor and good sensor. We have that $m \gg n$, so we must have more measurements than variables.

We're going to study a way to invert $z = Hx + e$, and get x from this.

Many possible estimates $\hat{x}$ can be given. A good estimate is given by one that, on average, is equal to the real value x , taking the name of centered estimator. This means that by doing several estimations and taking the average, we get the real value.

Another property is the efficiency of estimator. An efficient estimator is one that gives us results that are very close to the real value, and again here we have a probability distribution. The estimation error, $\hat{x} - x$, will tell us how large is the curve.

Let's focus on an estimation algorithm called Weighted Least Square Estimator (WLS). Under the assumption of zero mean Gaussian measurement noise, the WLS is efficient.

The intuition is that we have a vector of measurements. We know that:

$$z = \begin{pmatrix} z_1 \\ \vdots \\ z_m \end{pmatrix} = \begin{pmatrix} * & \dots & * \\ \vdots & H & \vdots \\ * & \dots & * \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$$

We have that measurements are known, and also the big matrix H is known. We need to find a good estimate for x , $\hat{x}$.

We cannot invert simply H , as $m \gg n$. Let's take the example:

$$\begin{pmatrix} 1 \\ 1.2 \\ 0.3 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 3 \\ 2 & 5 & 6 \\ 0.1 & 3 & 7 \\ 1 & 3 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

Since we know that $z = Hx + e$, we expect z to be very close to Hx . Intuitively, a good estimate is one such that $H \cdot \hat{x}$ is very close to z . This is called estimated measurement, $\hat{z} := H\hat{x}$. The goal is thus to find the estimate $\hat{x}$ that minimizes the distance between z and $H \cdot \hat{x}$ ($\hat{z}$). We define $r = z - \hat{z}$ to be the residual.

It will be a minimization problem. We know that z is a vector, and $\hat{z}$ is a vector as well. The difference between the two is the residual. This goal expressed in terms of residual is: Find $\hat{x}$ such that r is as small as possible, so as close as possible to zero vector. We have that:

$$r = \begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{pmatrix}$$

To translate this mathematically, the estimate

$$\hat{x} = \underset{x}{\operatorname{argmin}} \left(\underbrace{\sum_{i=1}^m (r_i)^2}_{\text{target function}} \right) = \underset{x}{\operatorname{argmin}} \left(\sum_{i=1}^m \underbrace{(z_i - H_i x)}_{r_i} \right).$$

$$\hat{x}_{LS} = \underset{x}{\operatorname{argmin}} \left(\sum_{i=1}^m (z_i - H_i x)^2 \right) \quad \text{unweighted least square estimation}$$

It's the unweighted least square estimation, as every residual count the same.

How do we weigh these residuals? Depends on the sensor. If z_i comes from an excellent sensor, then the value will have a big weight. Otherwise, smaller ones. So:

$$\hat{x}_{WLS} = \underset{x}{\operatorname{argmin}} \left(\sum_{i=1}^m \frac{1}{\sigma_i^2} (z_i - H_i x)^2 \right)$$